XM · CLASE 12

Soluciones de Analítica con Databricks

Clase 12: Orquestación con Lakeflow Jobs: el pipeline que se cuida solo

Docente: Manuela Larrea Gómez

Agenda de la clase 12

2 h · dónde vive cada cosa (con un caso del grupo), Genie, y el pipeline diario como un job que detecta sus propias fallas

01	20 MIN	**Ordenar el lakehouse: un caso del grupo** Catálogo, esquema y objeto · dónde van modelos, MLflow, telemetría y lo que escribe una app · cada consumidor, su capa · qué funciona en la industria
02	10 MIN	**Genie en vivo (pendiente de la clase 11)** La misma pregunta sin y con instrucciones · leer el SQL que genera
03	10 MIN	**Receso**
04	15 MIN	**Lakeflow Jobs** DAG, parámetros, task values, If/else, Run if, reintentos, disparadores, alertas, Repair run · la falla que no falla
05	60 MIN	**En acción: notebook guiado + lab · cierre** El job diario como código · tres corridas: normal, archivo que no llegó, reparación · tag s12

Antes de orquestar: ¿dónde vive cada cosa y por qué?

La pregunta del grupo. Tres reglas, y todo lo demás se deriva de ellas

1 · El contenedor dice quién y dónde

Catálogo = área + ambiente. Esquema = producto + capa. Objeto = entidad + grano. El ambiente nunca va en el nombre de la tabla; la versión nunca va en el nombre (para eso están Delta y el registro).

2 · Cada capa tiene un consumidor

raw: la ingesta. cleansed: el ingeniero. curated: el negocio (Power BI, Genie, apps). ml: el entrenamiento y el registro. monitoring: operación. Si no sabes quién consume una tabla, no sabes dónde va.

3 · Un solo escritor por tabla

Una tabla tiene un solo escritor: un job con su service principal. Los demás leen, idealmente por una vista. Lo que escribe una persona o una app va a su propio lugar.

Unity Catalog gobierna por niveles: permisos, linaje, etiquetas y almacenamiento se heredan de catálogo a esquema a objeto. Un buen orden hace que gobernar sea conceder 3 permisos, no 300.

Nivel	Convención	Ejemplo XM (curso)
Catálogo	área + ambiente	dev · qa · prod
Esquema	capa_dominio o producto_capa	gold_energia · <producto>_curated

Qué ha funcionado en la industria

Nada de esto es preferencia personal: es lo que recomiendan Databricks y los equipos que operan a escala

Unity Catalog: catálogos por ambiente o unidad de negocio

Grupos dueños de los objetos de producción, no personas. Los jobs de producción corren como service principal: "si usuarios ejecutan jobs que escriben en producción, se arriesga sobrescribir datos por accidente".

MLOps Stacks: deploy code, not models

Plantilla oficial de proyecto: código ML, recursos como código (bundles) y CI/CD. Se promueve el código que entrena; cada ambiente entrena su modelo.

Delta para la verdad analítica, Lakebase para la app

"Delta guarda la verdad analítica; Lakebase guarda el comportamiento operacional". No usar Delta como almacén directo del estado de una app que escribe con frecuencia.

Medallion y datos como producto

Cada capa con dueño, contrato y consumidor; la capa de consumo es un producto, no un depósito de resultados intermedios (data mesh, Dehghani).

Fuente: docs.databricks.com: Unity Catalog best practices · MLOps Stacks · developers.databricks.com: Lakebase vs. Delta tables · Z. Dehghani, "How to Move Beyond a Monolithic Data Lake to a Distributed Data Mesh", martinfowler.com, 2019

Genie en vivo: la misma pregunta, sin y con instrucciones

Pendiente de la clase 11. Espacio XM · pronóstico de demanda sobre v_observado_vs_estimado

1

Sin instrucciones

¿Cuál fue el WAPE del champion y del challenger en julio? → Show code

2

Con instrucciones

La misma pregunta en el espacio con la definición de WAPE → Show code

3

Una pregunta nueva

¿En qué mercados el champion tiene más error? En barras

4

Para el caso de ellos

Genie sobre curated (vistas con comentarios), nunca sobre cleansed

```
-- Instrucción clave del espacio
WAPE = sum(error_abs_kwh)
     / sum(abs(demanda_observada_kwh))
solo filas con demanda_observada_kwh no nula.
"El modelo" = alias 'champion'.
Reporta el WAPE en % con un decimal.
```

Genie es tan bueno como lo que se le enseña. La regla: leer siempre el SQL que generó, no la respuesta.

RECESO

10 minutos

Al volver: Lakeflow Jobs · el pipeline diario como código · la falla que no falla

Mientras tanto: Git folder en main → Pull · rama feature/c12-jobs · abre clases/c12-jobs/notebooks/lab12_orquestacion_jobs

iData
Global Data & AI

De ocho notebooks a un job: por qué no basta con programar una hora

Clases 4 a 11: cada paso funciona. Lo que falta es lo que pasa entre ellos y cuando algo sale mal

Pregunta de operación	Notebook programado (cron)	Lakeflow Job
¿En qué orden y qué depende de qué?	Lo sabe quien lo escribió	DAG explícito: depends_on
Si falla el paso 3, ¿qué pasa con el 4?	Corre igual, sobre datos viejos	Run if: no corre, o corre un control
Falla transitoria a las 6:00	Alguien lo relanza a mano	Reintentos donde es seguro
¿Llegó el archivo?	No se sabe	Disparador por llegada de archivo; If/else por task value
Falló a medias	Se relanza todo	Repair run: solo lo fallido y sus dependientes
¿Quién se entera?	Nadie, hasta que preguntan	Alerta por correo, Slack, Teams o webhook
¿Con qué identidad y qué código corrió?	La de quien lo programó, la copia del notebook	Run as service principal; código de un commit (Git o bundle)

Anatomía del job diario

[xm-demanda] diario · <usuario> — parámetros: usuario · fecha_proceso · forzar · umbral_cuarentena



Parámetros del job

Llegan a todas las tareas como widgets. En producción, dinámicos: {{job.start_time.iso_date}}.

Task values

Salidas pequeñas: bronce publica filas_nuevas y procesar; la condición las lee.

Run if

All succeeded (defecto), All done, None failed, At least one failed…

Reintentos y timeout

Solo donde reintentar es seguro. Timeout en todas: colgarse también es fallar.

Disparadores

Calendario con zona horaria, llegada de archivo, actualización de tabla, continuo.

Notificaciones

Al fallar, al terminar o si supera la duración esperada.

Cómputo e identidad

Serverless; Run as: service principal en producción.

Repair run

Solo lo fallido y sus dependientes, en la misma corrida.

Control de flujo: decidir en el job, no en el notebook

Task values + If/else + Run if: el job sabe qué hacer cuando no hay datos nuevos

notebooks de tareas

```
# t1_bronce (dbutils.jobs.*): publica lo que encontró
taskValues.set(key="filas_nuevas", value=nuevas)
taskValues.set(key="procesar", value=procesar)

# t6_chequeo: lo lee para su mensaje
taskValues.get(taskKey="bronce", key="filas_nuevas")
```

Run if	La tarea corre si…
All succeeded (defecto)	todas las anteriores terminaron bien
All done	terminaron, como sea (el chequeo)
None failed	ninguna falló
At least one failed	algo falló: limpieza o aviso
Ojo: todas excluidas	la tarea también se excluye

xm_demanda/orquestacion/job.py

```
hay_datos_nuevos: condition_task
  op: EQUAL_TO
  left: {{tasks.bronce.values.procesar}}
  right: si
calidad: depends_on hay_datos_nuevos
  outcome: true
```

For each

Una tarea que se repite con parámetros distintos: por ejemplo, entrenar un modelo por mercado de comercialización. Y tareas deshabilitadas para saltar un paso sin borrarlo.

Política de fallas: reintentar solo lo que es seguro reintentar

La pregunta no es "¿falló?", sino "si lo vuelvo a correr, ¿el resultado es el mismo?" — TODO 1 del lab

Tarea	¿Idempotente?	Reintentos	Timeout	Run if	Razón
bronce	sí (checkpoint)	0	30 min	All succeeded	Si falla, casi siempre es la fuente: reintentar no la arregla
calidad	sí (overwrite)	0	30 min	condición true	Si supera el umbral de cuarentena, reintentar da lo mismo
plata · oro	sí (MERGE)	0	30 min	All succeeded	Fallas de datos o de contrato: hay que mirarlas
inferencia	sí (MERGE)	2 · cada 1 min	30 min	All succeeded	Las fallas transitorias (carga del modelo, red) viven aquí
chequeo	sí (solo lee)	0	10 min	ALL_DONE	Un control que solo corre cuando todo salió bien no controla nada

Un reintento sobre una escritura con append duplica datos. La idempotencia de las clases 6, 7 y 11 era el requisito para poder orquestar.

La falla más peligrosa es la que no falla

27 de julio: el archivo del 26 no llegó. Todas las tareas que corren terminan "bien"

Sin chequeo

bronce ✓ 0 filas → condición ✓ false → calidad, plata, oro, inferencia: excluidas

El job termina en VERDE. El tablero muestra el pronóstico de ayer como si fuera de hoy, y le falta el 2 de agosto. Nadie se entera hasta que planeación pregunta.

Con chequeo (ALL_DONE + depende de bronce)

```
Pronóstico NO publicable:
- Oro llega hasta 2026-07-25 y para el
  proceso del 2026-07-27 se esperaba
  2026-07-26: 1 día(s) de atraso.
- Falta el pronóstico del 2026-08-02.
→ correo de alerta al dueño del job
```

Si el chequeo dependiera solo de inferencia, se excluiría con la rama y el job quedaría en verde. Reglas en src/…/chequeo.py, con tests. Luego: Repair run desde bronce.

Cuándo arranca, quién se entera, y quién orquesta a quién

En Free Edition: máximo 5 tareas simultáneas por cuenta → la cola del job espera su turno

Disparador	Úsalo cuando…
Calendario (cron + zona)	la fuente publica a hora fija
Llegada de archivo	la hora varía: arranca cuando llega
Actualización de tabla	otro equipo produce la entrada
Continuo	streaming: siempre encendido
API / otro job	lo dispara un sistema externo

Para XM: el pronóstico alimenta la planeación del día siguiente. La decisión de negocio es qué hacer si la fuente llega tarde: esperar, publicar marcado como provisional, o alertar. El job la hace explícita.

Azure Data Factory o Airflow

Cuando hay que coordinar sistemas fuera de Databricks (SAP, bases on-premise). Llaman al job por API: el job sigue siendo la unidad dentro de Databricks.

Notificaciones con criterio

Al fallar, a un buzón del equipo, no a una persona. Aviso por duración: si el job que dura 8 min lleva 40, algo pasa aunque no haya fallado.

max_concurrent_runs = 1 · cola activada

Lab 12: el pipeline diario como código, y tres corridas

clases/c12-jobs/notebooks/lab12_orquestacion_jobs · widget usuario · rama feature/c12-jobs

1

TODO 1: politica()

src/xm_demanda/orquestacion/job.py: reintentos en inferencia, ALL_DONE en chequeo

2

Crear el job (idempotente)

Si existe, jobs/reset; si no, jobs/create. Mismo diccionario que el YAML de la clase 13

3

Corrida 1 · día normal

fecha_proceso 2026-07-26, forzar si → todo en verde

4

Corrida 2 · no llegó el archivo

fecha_proceso 2026-07-27, forzar no → rama excluida, chequeo en rojo, correo

5

Corrida 3 · Repair run

Llega el archivo tarde → repair desde bronce → oro hasta 26-jul, pronóstico hasta 2-ago

Criterios de aceptación

- politica() con sus tests en verde
- Job con 7 tareas y chequeo ALL_DONE
- Una corrida exitosa y una reparada
- Oro hasta 26-jul; pronóstico hasta 2-ago
- Dos preguntas; verificar() en OK; PR en verde

Si la corrida queda en cola

Es la cuota de 5 tareas simultáneas de la cuenta, compartida por el grupo. No es un error: espera su turno.

Duración: 40 min guiado + 20 min solo

Así sería en Azure: el mismo job, desplegado y gobernado

Hoy (Free Edition)	En XM (Azure Databricks)
Job creado desde un notebook con la API	resources/job_diario.yml en un bundle, desplegado por CI/CD (clases 13 y 14)
Notebooks del Git folder personal	Código de un commit: Git source o el bundle desplegado; nunca una copia editable
Corre como el usuario	Run as: service principal por ambiente; dueño del job: un grupo
Calendario pausado, parámetro fecha fijo	Disparador por llegada de archivo o calendario; fecha = {{job.start_time.iso_date}}
Correo al usuario	Buzón del equipo, Teams o webhook a la herramienta de incidentes; aviso por duración
5 tareas simultáneas por cuenta	Cómputo serverless o clusters de job por tarea; límites por espacio de trabajo

Lo que no cambia: DAG explícito, política de fallas por tarea, control final que corre siempre y Repair run en vez de relanzar todo.

Resumen de la clase 12 y preparación para la clase 13

Logros de hoy

- **Cada cosa en la capa de su consumidor**
  Y quien escribe, separado de quien lee
- **El job ordena; src implementa**
  Notebooks delgados, lógica probada
- **Reintentar solo lo idempotente**
  Política de fallas por tarea
- **El chequeo corre siempre**
  La falla silenciosa se vuelve visible
- **Repair run**
  Solo lo fallido, en la misma corrida

Próxima clase

- **Asset Bundles**
  databricks.yml: el job de hoy como YAML
- **Ambientes dev / qa / prod**
  Targets, variables y service principals
- **Deploy desde la CLI**
  Y un deploy con una variable faltante
- **Traer a la clase 13**
  Lab 12 terminado y CLI de Databricks instalada



iData
Global | Data & AI

Gracias

Nos vemos en la clase 13: Asset Bundles y ambientes

DOCENTE

Manuela Larrea Gómez